Esercizio 2

Introduzione

L'algoritmo **Edit Distance (o distanza di Levenshtein)** è un metodo fondamentale in informatica che calcola il numero minimo di operazioni necessarie per trasformare una stringa in un'altra.
Nel nostro caso, abbiamo implementato due versioni dell'algoritmo: una ricorsiva semplice e una ottimizzata mediante programmazione dinamica.
Le operazioni considerate sono l'inserimento e la cancellazione di un carattere, con la sostituzione gestita implicitamente come combinazione di queste operazioni.

Funzione edit_distance

La funzione **edit_distance** implementa una versione ricorsiva dell'algoritmo. Essa calcola la distanza di edit tra le stringhe s1 e s2 con una semplice ricorsione, senza l'uso di memorizzazione intermedia.
La funzione utilizza due operazioni principali:

- **cancellazione di un carattere**
- **inserimento di un carattere**

La funzione esplora ricorsivamente tutte le possibili operazioni, confrontando il risultato e restituendo il minimo tra loro.
Questa versione, purtroppo, è molto inefficiente a causa della ripetizione dei calcoli per le stesse sottostringhe.
In pratica, ogni chiamata ricorsiva comporta l'esecuzione di molti calcoli ridondanti.

Funzione edit_distance_dyn (versione ottimizzata)

La versione ottimizzata dell'algoritmo utilizza la programmazione dinamica. Invece di calcolare ricorsivamente i sottoproblemi, vengono memorizzati i risultati parziali in una matrice mat, che memorizza la distanza di edit tra tutte le possibili sottostringhe di s1 e s2.
La matrice ha dimensioni (m+1) x (n+1), dove m e n sono le lunghezze delle due stringhe.
Dopo aver inizializzato la matrice, per ogni coppia di caratteri nelle stringhe, si calcola la distanza di edit utilizzando la relazione di ricorrenza:

- Se i caratteri sono uguali, non è necessaria alcuna modifica, quindi il valore viene copiato dalla cella diagonale precedente.
- Se i caratteri sono diversi, si calcola il valore minimo tra due opzioni: inserimento e cancellazione.

Osservazioni sulla versione dinamica:

Questa versione è decisamente più efficiente rispetto alla versione ricorsiva, poiché evita calcoli ripetitivi e ridondanti.

Grazie alla memorizzazione dei risultati intermedi nella matrice mat, la soluzione viene ottenuta in tempo O(m*n), dove m e n sono le lunghezze delle due stringhe. Questo rende l'algoritmo molto più scalabile, specialmente per stringhe di dimensioni elevate.

Problema avuto duranto lo sviluppo:

La versione commentata di edit_distance_dyn_rec è una versione dinamica con memorizzazione in cui i calcoli vengono memorizzati in una matrice mat e vengono passati come argomento alle chiamate ricorsive. Tuttavia, rispetto alla versione ottimizzata che abbiamo esaminato, il codice presenta alcune inefficienze. La gestione della matrice come parametro passato tra le funzioni aumenta la complessità complessiva del codice.
In particolare, la creazione della matrice e la gestione dei casi base erano meno efficienti rispetto alla versione che popola la matrice in modo iterativo.

Conclusioni

Il codice presentato mostra due implementazioni dell'algoritmo di Edit Distance. La versione ricorsiva, sebbene corretta, è inefficiente per stringhe lunghe a causa della ripetizione dei calcoli. La versione ottimizzata, basata sulla programmazione dinamica, risolve il problema in modo più efficiente, utilizzando una matrice per memorizzare i risultati intermedi e ridurre la complessità temporale. In più elimina completamente i calcoli ridondanti memorizzando i risultati intermedi.